RAPPORT D'INGÉNIERIE DES DONNÉES · PROJET 1

Santé mondiale : les ODD 3 en Afrique

Un entrepôt PostgreSQL fiable des indicateurs de l'OMS : ingestion, qualité, UPSERT et analyse des cibles 2030

LUMU BAMANA MANASSE

Data engineer

Date	Octobre 2026
Version	1.0.0 (livrée et recettée)
Client	Initiative Baobab Santé (ONG fictive) · Mission IBS-2026-001
Dépôt GitHub	https://github.com/MediJr/who-sdg3-health-etl
Langue	Français · *English version available in the repository*

Scénario fictif, données réelles : l'ONG cliente et ses interlocuteurs sont inventés pour simuler une mission professionnelle complète. Les données proviennent de l'Observatoire mondial de la santé de l'OMS (Global Health Observatory), extraites le 22 octobre 2026. Ce projet n'est ni commandé, ni validé, ni affilié à l'Organisation mondiale de la santé.

Fiche du document

Projet	Projet 1 du portfolio : entrepôt PostgreSQL des indicateurs de santé de l'OMS
Mission	IBS-2026-001 : « Suivre l'ODD 3 en Afrique avec les données de l'OMS »
Client	Initiative Baobab Santé, ONG de santé maternelle et infantile (fictive), à l'attention de Dr Mariam Koné
Auteur	Lumu Bamana Manasse, data engineer
Période de réalisation	Octobre 2026
Données	API GHO de l'OMS, lot du 22 octobre 2026
Statut	LIVRÉ RECETTÉ
Code source	https://github.com/MediJr/who-sdg3-health-etl

Historique des versions

Version	Date	Objet
0.1	22/10/2026	Cadrage : brief, compte rendu, cahier des charges
0.9	22/10/2026	Pipeline, tests, analyses, corrections (qualité, performance)
1.0.0	22/10/2026	Recette sur environnement neuf, rapport final (français et anglais)

Guide de lecture

Conseil d'administration	1. Résumé exécutif et 11. Résultats et recommandations
Responsable suivi-évaluation	2. Cadrage, 6. Modèle de données, 7. Qualité, 12. Limites
Ingénieur reprenant le projet	4. Architecture, 5. Pipeline, 9. Exploitation, 10. Tests, annexes

Table des matières

Fiche du document 2

1 Résumé exécutif 4

2 Contexte et cadrage 5

2.1 Les vrais problèmes du client 5

2.2 Questions business 5

2.3 Exigences techniques 6

2.4 Périmètre 6

3 Sources de données 6

4 Architecture de la solution 7
4.1 Choix techniques 8
4.2 Ce qui change par rapport au projet 0 8

5 Conception du pipeline 8
5.1 Extraction 8
5.2 Transformation et contrôles de qualité 9
5.3 Chargement : staging et UPSERT 9

6 Modèle de données 10

7 Qualité des données 10
7.1 Couverture 10
7.2 Calibrage des seuils : une leçon sur les données réelles 10

8 Sécurité et gouvernance 11

9 Exploitation 11
9.1 Commandes 11
9.2 Diagnostics de production 12

10 Tests et recette 12
10.1 Tests automatisés 12
10.2 Recette sur environnement neuf 13

11 Résultats et recommandations 13
11.1 Q1 · Où en sont les pays par rapport aux cibles de 2030 ? 13
11.2 Q2 · Quels pays atteindront les cibles d'ici 2030 ? 15
11.3 Q3 · Quel écart entre les ménages les plus pauvres et les plus riches ? 17
11.4 Q4 · Les moyens vont-ils de pair avec les résultats ? 18
11.5 Recommandations au conseil d'administration 19

12 Limites et dette technique 19

13 Retour d'expérience 19

14 Feuille de route 20

A Requêtes SQL livrées 20

B Décisions d'architecture (ADR) 20

C Glossaire 21

1 Résumé exécutif

Le besoin. L'ONG Initiative Baobab Santé suit chaque année la progression des pays africains vers les cibles de l'ODD 3. Son processus était manuel (téléchargements, copier-coller dans Excel), non reproductible, et avait déjà produit une erreur présentée au conseil d'administration : un tableau mélangeait plusieurs catégories de données sans que personne ne le voie.

La solution. Un pipeline Python alimente automatiquement un entrepôt **PostgreSQL** conteneurisé avec 9 indicateurs de l'OMS. Le grain est explicite, les contrôles de qualité mettent en quarantaine les lignes invalides, et un chargement **UPSERT** intègre les révisions de l'OMS sans jamais créer de doublon. L'équipe du client interroge la base en SQL.

129 368 observations de l'OMS	**9** indicateurs ODD 3	**386** lieux (pays, régions, monde)	**22 / 22** tests automatisés
0 / 0 insertions / mises à jour à la relance	**10 s** rechargement complet	**×15** gain de performance après diagnostic	**0** donnée personnelle

Principaux résultats pour la région africaine de l'OMS (47 pays)

— **Q1 · Cibles 2030 :** seuls **5 pays** sur 47 ont atteint la cible de mortalité des moins de 5 ans (25 ‰), et 4 celle de la mortalité maternelle (70 pour 100 000). La région reste loin derrière la moyenne mondiale.

— **Q2 · Rythme : aucun** pays hors cible n'est en voie d'atteindre l'objectif de mortalité infantile d'ici 2030. Le pays médian progresse de 3,0 % par an, alors qu'il lui faudrait 12,4 % : **quatre fois plus vite**.

— **Q3 · Inégalités :** un enfant des 20 % de ménages les plus pauvres a **1,7 fois plus de risque** de mourir avant 5 ans (jusqu'à 2,5 fois au Nigeria). L'écart absolu se réduit, mais l'écart relatif se creuse depuis 2000.

— **Q4 · Moyens :** les **accouchements assistés** sont le moyen le plus lié à une faible mortalité (corrélation de $-0{,}75$).

Ce que démontre ce projet, côté ingénierie : conteneurisation d'une base PostgreSQL, gestion des secrets hors du code, modèle en couches (staging, core, ops), clé métier complète avec `NULLS NOT DISTINCT`, chargement en masse par `COPY` suivi d'un UPSERT idempotent et mesurable, contrôles de qualité avec quarantaine, journal des exécutions, tests unitaires et d'intégration, et deux diagnostics de production : une panne de Docker et une connexion de 130 secondes ramenée à zéro.

2 Contexte et cadrage

2.1 Les vrais problèmes du client

L'analyse du brief a montré que le problème du client était d'abord un problème d'**ingénierie des données**, et seulement ensuite d'analyse :

Symptôme décrit par le client	Cause technique
« Nous téléchargeons les tableaux à la main »	Aucun processus reproductible ni traçable
« Les chiffres ne concordent jamais d'une année sur l'autre »	L'OMS révise ses estimations, et personne ne garde la trace des révisions
« Un tableau mélangeait plusieurs catégories »	**Erreur de grain** : une même valeur pays-année existe par sexe, par âge et par quintile de richesse

Table 1 – Du symptôme à la cause

2.2 Questions business

#	Question	Décision éclairée
Q1	Où en sont les pays de la région africaine par rapport aux cibles ODD 3 de 2030 ?	Diagnostic présenté au conseil
Q2	À leur rythme actuel, quels pays atteindront ces cibles d'ici 2030 ?	Priorisation des pays à financer
Q3	Quel est l'écart de mortalité infantile entre les ménages les plus pauvres et les plus riches ?	Ciblage des programmes
Q4	Les moyens du système de santé vont-ils de pair avec de meilleurs résultats ?	Choix des leviers d'action

Table 2 – Questions retenues lors du cadrage

2.3 Exigences techniques

Exigence	Réponse technique
Base accessible en SQL à plusieurs utilisateurs	PostgreSQL 17 dans Docker
Relançable à tout moment, sans doublon	UPSERT sur une clé métier complète
Dernière valeur publiée, et date de révision	`last_changed_at`, `last_run_id`
Grain explicite	Clé `UNIQUE NULLS NOT DISTINCT` sur 7 colonnes
Contrôles avant chargement	9 règles et une table de quarantaine
Mots de passe hors du code	Fichier `.env` exclu de Git
Tests automatisés	`pytest` : 17 tests unitaires et 5 tests d'intégration

TABLE 3 – Exigences du cahier des charges et réponses apportées

2.4 Périmètre

La base contient les 9 indicateurs pour **tous les pays et toutes les régions** publiés par l'OMS : elle est réutilisable au-delà de cette mission. Les analyses portent sur la **région africaine de l'OMS** (47 pays avec données), avec la valeur mondiale comme référence.

3 Sources de données

Ressource GHO	Contenu	Volume (lot du 22/10)
`/api/{indicateur}`	Observations : valeur, bornes d'incertitude, date de mise à jour	129 368 lignes, 9 indicateurs
`/api/Indicator`	Libellés officiels des indicateurs	9 lignes
`/api/DIMENSION/{type}/DimensionValues`	Libellés des lieux et des ventilations	386 lieux, 174 ventilations

TABLE 4 – Ressources de l'API de l'OMS utilisées (protocole OData, sans clé d'API)

Code	Indicateur	Lignes	Cible 2030
MDG_0000000007	Mortalité des moins de 5 ans (‰)	64 510	≤ 25
WHOSIS_000001	Espérance de vie à la naissance	14 112	–
WHOSIS_000002	Espérance de vie en bonne santé	14 112	–
WHOSIS_000003	Mortalité néonatale (‰)	12 128	≤ 12
MDG_0000000026	Mortalité maternelle (pour 100 000)	7 878	< 70
WHS4_100	Couverture vaccinale DTP3 (%)	5 337	90 %
GHED_CHE_pc_US_SHA2011	Dépenses de santé par habitant (USD)	4 849	–
HWF_0001	Médecins pour 10 000 habitants	3 693	–
MDG_0000000025	Accouchements assistés (%)	2 749	–

TABLE 5 – Indicateurs collectés

Découverte clé de l'exploration : la dimension cachée

Pour le Sénégal en 2020, la mortalité des moins de 5 ans apparaît **8 fois avec 8 valeurs différentes**, de 27,9 à 61,2 ‰. Ce ne sont pas des doublons : la colonne `Dim3` porte le **quintile de richesse** (61 ‰ chez les 20 % de ménages les plus pauvres, 28 ‰ chez les 20 % les plus riches). C'est exactement l'erreur commise par le client l'année précédente. Cette découverte a fixé le **grain** de la table de faits et a fait naître la question Q3.

4 Architecture de la solution

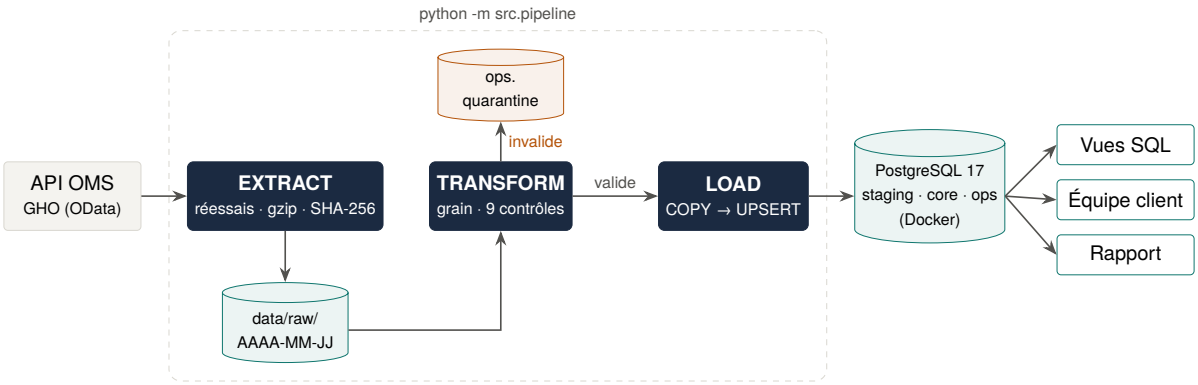


FIGURE 1 – Architecture : une commande collecte, contrôle et charge les données dans PostgreSQL

4.1 Choix techniques

Besoin	Choix	Justification
Base de données	PostgreSQL 17	Multi-utilisateur, contraintes fortes, UPSERT natif, `NULLS NOT DISTINCT` (ADR 001)
Exécution de la base	Docker Compose	Même base sur tous les postes, en une commande, sans installation manuelle
Pilote	psycopg 3	Pilote moderne, `COPY` pour le chargement en masse
Secrets	python-dotenv + `.env`	Aucun mot de passe dans le code ni dans Git
Réessais HTTP	`urllib3.Retry`	Attente croissante, respect de l'en-tête `Retry-After`
Tests	pytest	Tests unitaires sans base, tests d'intégration sur une base dédiée

Table 6 – Stack technique (en plus de Python, pandas et matplotlib, déjà utilisés au projet 0)

4.2 Ce qui change par rapport au projet 0

	Projet 0	Projet 1
Base	DuckDB (fichier)	PostgreSQL (serveur, conteneur)
Chargement	Rechargement complet	Staging + UPSERT, données conservées
Idempotence	Garantie par le remplacement	**Mesurée** : compteurs insérées / mises à jour / inchangées
Qualité	Contrôles a posteriori	Contrôles **avant** chargement, et quarantaine
Secrets	Aucun	Fichier `.env`
Tests	Recette manuelle	22 tests automatisés
Traçabilité	Manifeste	Manifeste avec SHA-256, et journal des exécutions en base

Table 7 – Progression du portfolio

5 Conception du pipeline

5.1 Extraction

— **Réessais confiés à** `urllib3.Retry` : jusqu'à 4 tentatives, avec des attentes de 2, 4, 8 et 16 s, sur les erreurs temporaires uniquement (429, 500, 502, 503, 504) et jamais sur une erreur 404.

— **Pagination OData suivie** (`@odata.nextLink`), et **échec explicite** si la réponse ne contient pas la clé `value` : mieux vaut planter que charger n'importe quoi.

— **Couche brute compressée** (gzip, 6,4 Mo pour 129 368 lignes) et **écriture atomique**.

— **Manifeste** avec l'empreinte SHA-256 de chaque fichier, qui prouve l'intégrité du lot.

5.2 Transformation et contrôles de qualité

Les ventilations sont rangées selon leur **type déclaré** (`Dim1Type`, `Dim2Type`…), jamais selon leur position : l'OMS peut placer le sexe en première ou en deuxième dimension selon l'indicateur. Neuf règles sont ensuite appliquées :

#	Règle (motif de quarantaine)
1	Indicateur hors référentiel
2	Valeur absente ou non numérique
3	Valeur hors bornes de vraisemblance (limites logiques, voir section 7)
4	Intervalle d'incertitude incohérent (borne basse > valeur, ou valeur > borne haute)
5	Année invalide
6	Lieu absent du référentiel
7	Ventilation absente du référentiel
8	Type de ventilation inconnu
9	Doublon de clé métier dans le lot

Table 8 – Règles de qualité appliquées avant chargement

5.3 Chargement : staging et UPSERT

Le lot est copié en masse dans `staging.observation` (`COPY`, 0,9 s), puis fusionné dans `core.fact_observation` par une seule requête :

```
INSERT INTO core.fact_observation AS f (...)
SELECT ... FROM staging.observation
ON CONFLICT ON CONSTRAINT uq_observation_business_key DO UPDATE SET
    value = EXCLUDED.value, value_low = EXCLUDED.value_low, value_high = EXCLUDED.value_high,
    last_changed_at = now(), last_run_id = EXCLUDED.last_run_id
WHERE (f.value, f.value_low, f.value_high)
      IS DISTINCT FROM (EXCLUDED.value, EXCLUDED.value_low, EXCLUDED.value_high)
RETURNING (xmax = 0) AS inserted      -- vrai : insérée, faux : mise à jour
```

— `ON CONFLICT` : si la clé métier existe déjà, on met à jour au lieu d'insérer.

— `WHERE … IS DISTINCT FROM` : on ne touche **que** les valeurs réellement révisées par l'OMS.

— `IS DISTINCT FROM` plutôt que <> : il compare correctement les `NULL`, ce que <> ne fait pas.

— Fusion, quarantaine et journal sont validés dans **une seule transaction**. Le journal des exécutions est ouvert à part, pour qu'un échec reste tracé.

6 Modèle de données

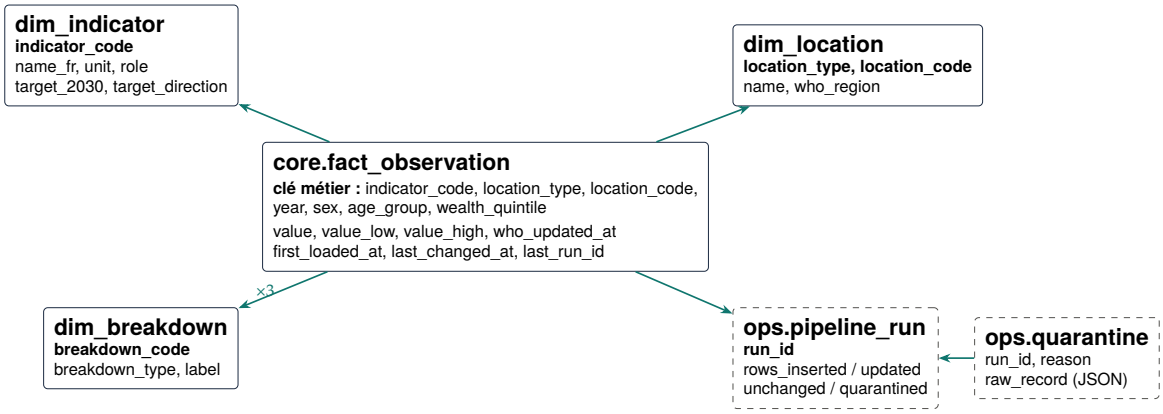


FIGURE 2 – Modèle de données (schémas core et ops ; les flèches représentent les clés étrangères)

> **Pourquoi NULLS NOT DISTINCT ?**
>
> Par défaut, en SQL, `NULL = NULL` n'est pas vrai : deux lignes « sans ventilation » ne sont donc pas considérées comme des doublons, et une contrainte d'unicité classique laisserait passer une infinité de copies. `UNIQUE NULLS NOT DISTINCT` (PostgreSQL 15 et plus) traite les `NULL` comme égaux, et ferme cette faille.

Vues d'analyse. `core.v_national` encapsule la règle de grain (deux sexes, sans ventilation par richesse) et `core.v_latest` donne la dernière année disponible. L'équipe du client n'a plus à connaître ces règles pour obtenir une valeur juste.

7 Qualité des données

7.1 Couverture

Les 9 indicateurs sont disponibles pour les **47 pays** de la région africaine ayant des données (sur 50 codes de pays et territoires). Les données sont récentes : 2023 à 2025 selon les indicateurs. Seule exception : les accouchements assistés, pour lesquels seuls 32 pays ont une donnée postérieure à 2020 (médiane : 2021).

7.2 Calibrage des seuils : une leçon sur les données réelles

Au premier passage, des seuils de vraisemblance « intuitifs » ont mis **25 lignes en quarantaine**. Leur examen a montré qu'il ne s'agissait pas d'erreurs, mais des traces chiffrées de crises humanitaires majeures :

Seuil initial	Lignes	Ce qu'étaient ces valeurs
Mortalité des moins de 5 ans ≤ 500 ‰	18	Cambodge 1975-1978 (régime khmer rouge), Soudan du Sud 1988, Irak 1942-1944
Mortalité maternelle ≤ 5 000	7	Rwanda 1994 (génocide), Soudan du Sud 1987-1998 (guerre civile)

Table 9 – Valeurs extrêmes mais réelles, mises en quarantaine à tort

Les seuils ont été remplacés par des **limites logiques** : un taux pour 1 000 naissances ne dépasse pas 1 000, un pourcentage pas 100 (ADR 003). Résultat : 129 368 **lignes valides, 0 en quarantaine**, et les contrôles continuent de bloquer les erreurs réelles, comme le prouvent les tests.

> **Leçon**
> Une règle de qualité se calibre avec la connaissance du domaine. La quarantaine a joué son rôle : isoler les valeurs suspectes pour qu'un humain les examine, au lieu de les supprimer en silence ou de les charger sans examen.

8 Sécurité et gouvernance

Thème	Mesure
Secrets	Identifiants PostgreSQL dans `.env` (exclu de Git). Modèle sans secret `.env.example` versionné. Mot de passe aléatoire généré localement.
Exposition réseau	Base accessible uniquement depuis la machine locale (`127.0.0.1:5433`)
Données personnelles	Aucune : les données de l'OMS sont agrégées à l'échelle des pays
Licence des données	Données de l'OMS utilisées selon sa politique d'utilisation des données, à des fins non commerciales, avec mention de la source
Traçabilité	Manifeste SHA-256, journal `ops.pipeline_run`, `last_run_id` sur chaque observation, ADR, journal de bord

9 Exploitation

9.1 Commandes

```
copy .env.example .env                          # puis choisir un mot de passe
docker compose up -d                            # démarre PostgreSQL (port 5433)
pip install -r requirements.txt
python -m src.pipeline                          # extraction + chargement + rapport
python -m src.pipeline --run-date 2026-10-22    # rejoue un lot brut, sans appel à l'API
```

```
pytest                                          # 22 tests (dont 5 sur une base dédiée)
```

9.2 Diagnostics de production

Deux incidents réels ont été diagnostiqués et résolus pendant la mission :

Incident	Diagnostic	Résolution
Docker Desktop plante au démarrage	Journal du backend : deux sockets Unix (`sailor-ingest.sock`, `engine.sock`) verrouillés depuis un arrêt brutal, un mois plus tôt	Dossiers renommés (rien supprimé). Docker recrée des dossiers propres.
Chargement en 156 s pour 129 368 lignes	Mesure étape par étape : `COPY` 0,9 s, UPSERT 2,8 s, mais **connexion 130 s**. `localhost` est résolu d'abord en IPv6 (`::1`), où Docker n'écoute pas.	Hôte `127.0.0.1` et `connect_timeout=10` : pipeline en **10 s** (×15)

TABLE 10 – Incidents résolus

> **Leçon**
>
> Mesurer au lieu de supposer : le goulot d'étranglement n'était ni dans le code Python ni dans PostgreSQL, mais dans la résolution du nom d'hôte.

10 Tests et recette

10.1 Tests automatisés

Fichier	Tests	Ce qui est vérifié
`test_extract.py`	4	Pagination OData, échec sur structure inattendue, écriture atomique, réessais limités aux erreurs temporaires
`test_transform.py`	13	Grain (quintiles distincts), ventilations rangées par type, chacune des règles de quarantaine, aucune ligne perdue
`test_load_integration.py`	5	Sur une vraie base dédiée : premier chargement, relance idempotente, révision détectée, quarantaine en base, contrainte d'unicité

TABLE 11 – 22 tests, sur un extrait réel de l'API (Sénégal, 2020)

Les tests ont trouvé un **vrai bug** : dans une colonne pandas mixte, `None` devient `NaN`, et `NaN` est « vrai » en Python. Une seule ligne à la ventilation inconnue envoyait alors **toutes** les lignes en quarantaine.

10.2 Recette sur environnement neuf

Copie des seuls fichiers publiables, nouvel environnement Python, `.env` recréé à partir du modèle, **base PostgreSQL vierge**, extraction réelle depuis l'API.

#	Critère	Résultat	
C1	Exécution de bout en bout, base comprise	Installation en 57 s, pipeline en 2 min 43 s (extraction réelle comprise)	✓
C2	Relance sans doublon ni modification	0 insérée, 0 mise à jour, 129 368 inchangées	✓
C3	Clé métier couvrant toutes les dimensions	Contrainte en base, 0 doublon, test d'intégration	✓
C4	Lignes invalides en quarantaine, jamais perdues	Invariant extraites = chargées + quarantaine	✓
C5	Aucun secret dans le dépôt	`.env` exclu, aucun mot de passe dans les fichiers versionnés	✓
C6	Suite de tests intégralement verte	22 / 22	✓
C7	Une requête et une réponse par question	Section 11	✓

Table 12 – Résultats de la recette (détails : `docs/livraison/2026-10-22-pv-recette.md`)

11 Résultats et recommandations

11.1 Q1 · Où en sont les pays par rapport aux cibles de 2030 ?

Indicateur	Cible	Pays à la cible	Médiane Afrique	Monde
Mortalité des moins de 5 ans (‰, 2024)	≤ 25	5 / 47 (11 %)	48,7	37,4
Mortalité néonatale (‰, 2024)	≤ 12	4 / 47 (9 %)	23,0	17,2
Mortalité maternelle (/100 000, 2023)	< 70	4 / 47 (9 %)	258,2	197,3
Couverture vaccinale DTP3 (%, 2025)	≥ 90	16 / 47 (34 %)	84	85

Table 13 – Situation par rapport aux cibles de 2030 (dernière année disponible)

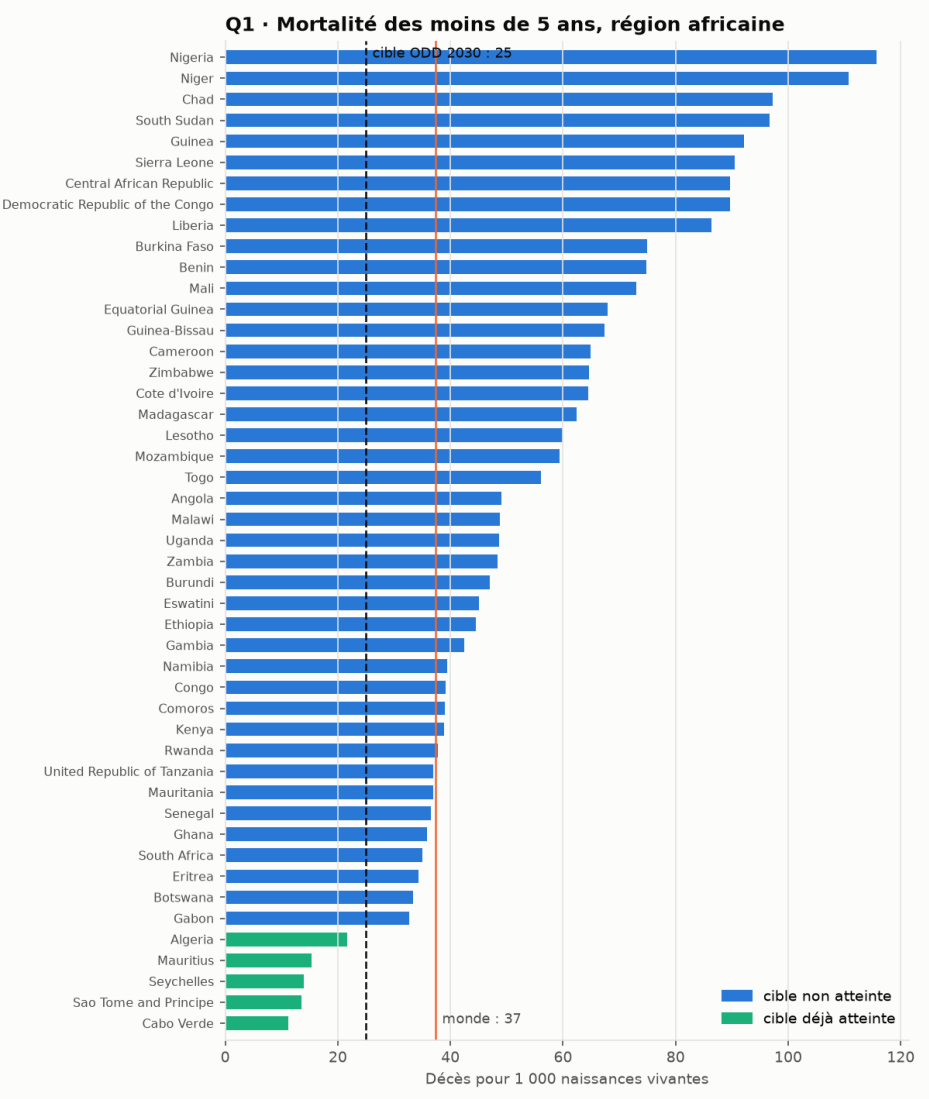


FIGURE 3 – Mortalité des moins de 5 ans par pays de la région africaine (2024)

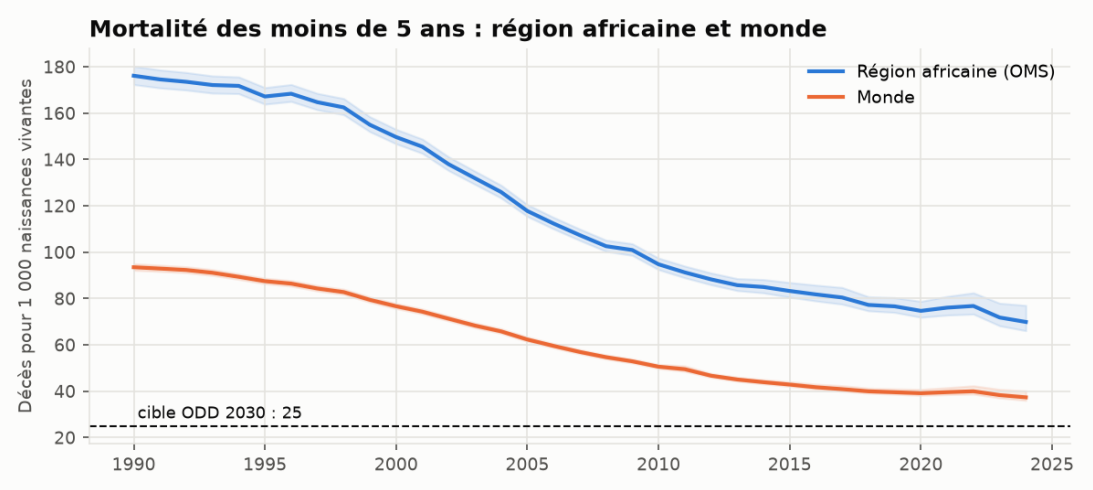


FIGURE 4 – Évolution depuis 1990, avec intervalles d'incertitude

La mortalité des moins de 5 ans de la région a été divisée par 2,5 depuis 1990, de 176 à 70 ‰. Elle reste près de deux fois supérieure à la moyenne mondiale. La valeur régionale (70 ‰) dépasse nettement la médiane des pays (48,7 ‰) : les pays les plus peuplés, comme le Nigeria (116 ‰), pèsent lourd dans le total régional.

11.2 Q2 · Quels pays atteindront les cibles d'ici 2030 ?

Méthode du **taux annuel moyen de réduction** (ARR), utilisée par le groupe interagences des Nations unies pour l'estimation de la mortalité de l'enfant : $\text{ARR} = \ln(v_{2015}/v_t)/(t - 2015)$, puis projection à 2030 en supposant que ce rythme se maintient.

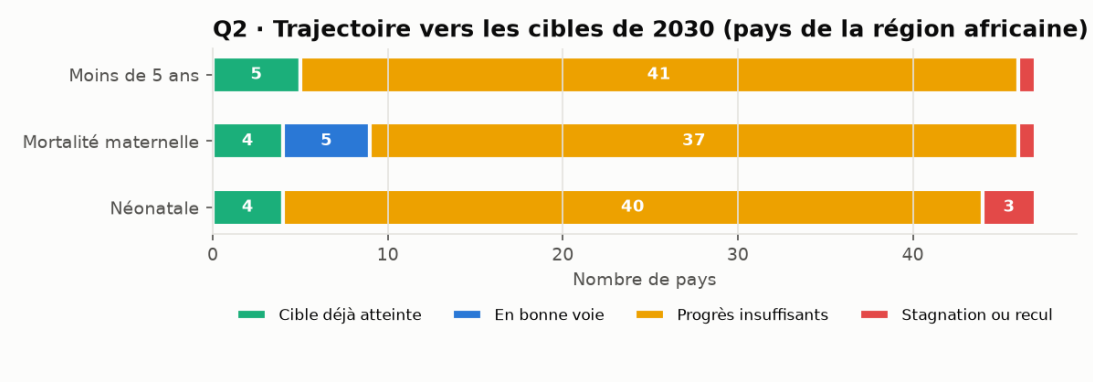


FIGURE 5 – Statut des pays de la région africaine par rapport à leur trajectoire

— **Mortalité des moins de 5 ans :** aucun pays hors cible n'est en bonne voie. Le pays médian réduit sa mortalité de **3,0 % par an** alors qu'il faudrait **12,4 %**. Le Zimbabwe recule (59,2 → 64,7 ‰ depuis 2015).

— **Mortalité maternelle :** c'est la seule cible où 5 pays sont en bonne voie : Eswatini, Mozambique, Namibie, Sao Tomé-et-Principe et Zambie, avec des réductions de 7 à 11 % par an.

Le Botswana recule.

— **Mortalité néonatale :** la plus lente (1,7 % par an). Elle stagne ou recule au Nigeria, en Afrique du Sud et au Zimbabwe.

11.3 Q3 · Quel écart entre les ménages les plus pauvres et les plus riches ?

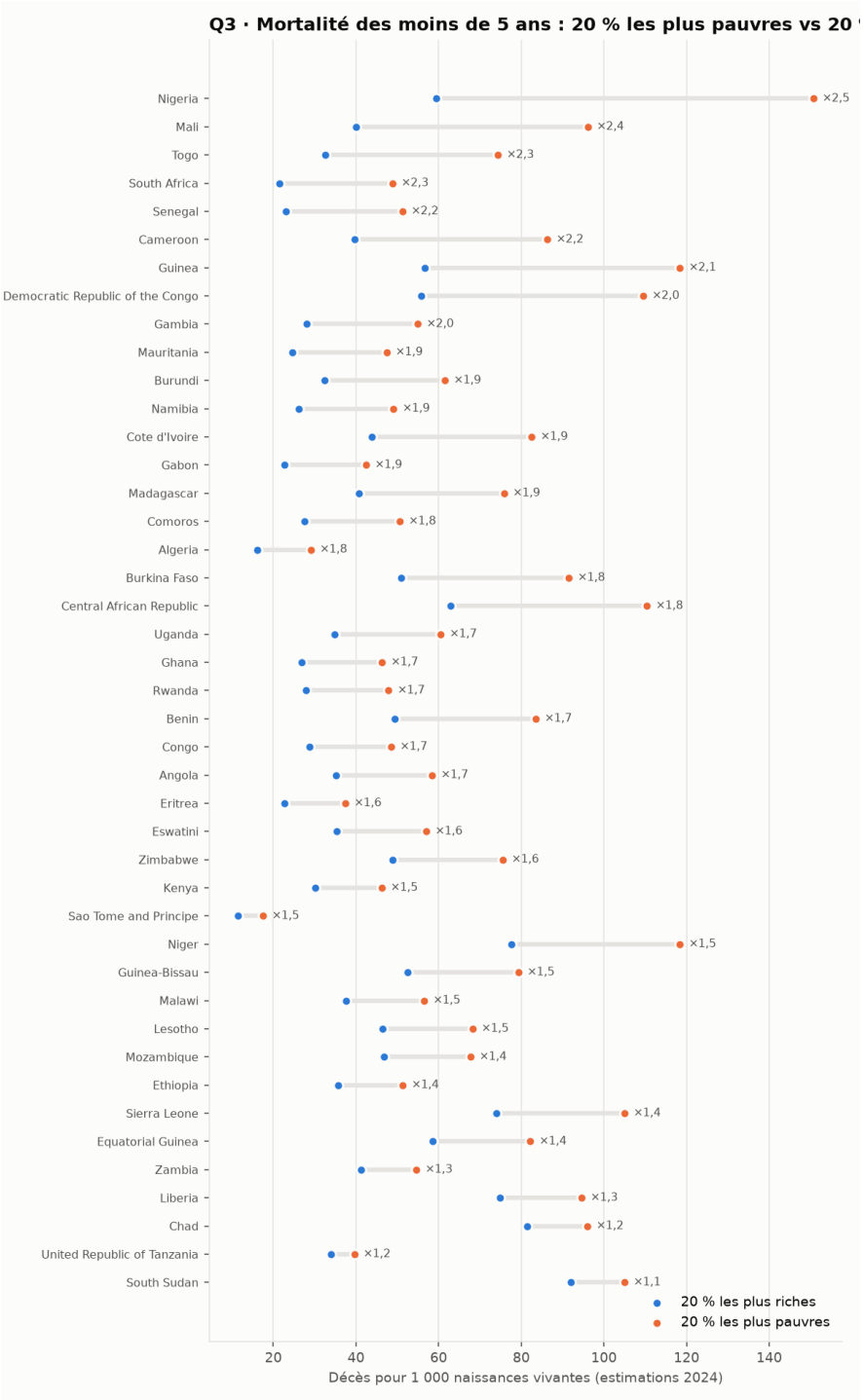


FIGURE 6 – Mortalité des moins de 5 ans : 20 % les plus pauvres contre 20 % les plus riches (2024)

Année	20 % les plus pauvres	20 % les plus riches	Écart absolu	Ratio
1990	186,6	118,8	63,3	1,56
2000	159,7	94,2	50,5	1,49
2015	74,4	46,3	28,2	1,67
2024	61,5	37,6	23,2	1,71

TABLE 14 – Valeurs médianes de 43 pays de la région africaine (décès pour 1 000 naissances vivantes)

Un enfant né dans les 20 % de ménages les plus pauvres a **1,7 fois plus de risque** de mourir avant 5 ans, et jusqu'à 2,5 fois au Nigeria et 2,4 fois au Mali. **L'écart absolu s'est réduit** de 63 à 23 ‰, mais **l'écart relatif se creuse depuis 2000** : les ménages aisés progressent proportionnellement plus vite.

11.4 Q4 · Les moyens vont-ils de pair avec les résultats ?

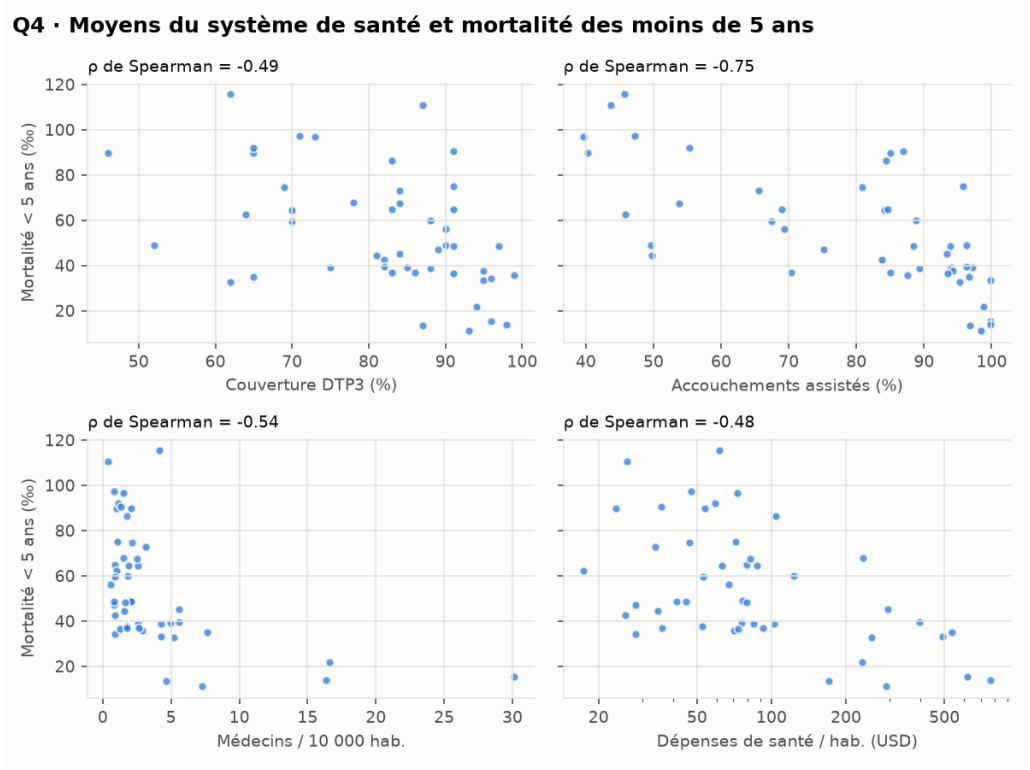


FIGURE 7 – Indicateurs de moyens et mortalité des moins de 5 ans (un point = un pays)

Tous les moyens sont associés à une mortalité plus faible, mais avec une intensité très différente : **accouchements assistés** ($\rho = -0{,}75$, sur 45 pays), médecins ($-0{,}54$), vaccination DTP3 ($-0{,}49$), dépenses de santé ($-0{,}48$). Il s'agit de **corrélations**, pas de preuves de causalité : ces moyens reflètent aussi le niveau de développement général des pays.

11.5 Recommandations au conseil d'administration

Pour Initiative Baobab Santé

1. **Prioriser la mortalité néonatale :** c'est la cible qui progresse le plus lentement (1,7 % par an). Les décès du premier mois représentent près de la moitié de la mortalité des moins de 5 ans (médianes : 23,0 sur 48,7 ‰).
2. **Cibler explicitement les ménages pauvres :** l'écart relatif se creuse. Un programme non ciblé profite d'abord aux ménages aisés.
3. **Miser sur l'accouchement assisté :** c'est le levier le plus fortement associé à une faible mortalité dans la région.
4. **Étudier les pays en bonne voie** sur la mortalité maternelle (Mozambique, Zambie, Eswatini…) pour en tirer des enseignements.
5. **Relancer le pipeline après chaque publication de l'OMS :** les révisions sont désormais intégrées et tracées automatiquement.

12 Limites et dette technique

Limite	Conséquence
Les données de l'OMS sont des estimations modélisées	Les intervalles d'incertitude sont stockés, mais pas encore affichés pour chaque pays
Projection linéaire du rythme (ARR)	Elle suppose que le rythme 2015-2024 se maintient jusqu'en 2030
Corrélations (Q4)	Elles ne prouvent aucune causalité
Noms de pays en anglais	L'API de l'OMS ne publie pas de libellés français
Dette technique assumée au niveau 1	
Seule la dernière valeur est conservée	L'historique complet des révisions sera introduit au projet 2
Suppressions de l'OMS non gérées	Une observation retirée par l'OMS resterait en base
Pas de planification	Exécution manuelle (orchestration au projet 3)
Pas d'intégration continue	Les tests sont lancés à la main (CI/CD au projet 3)

13 Retour d'expérience

— **Le grain d'abord :** 8 valeurs « identiques » cachaient une dimension. Comprendre le grain avant de modéliser évite l'erreur du client.

— **Un contrôle de qualité se calibre :** 25 valeurs « aberrantes » étaient des catastrophes histo-

riques réelles.

— **Mesurer au lieu de supposer :** 130 s perdues dans une résolution DNS, pas dans le code.

— **Les tests trouvent de vrais bugs :** le piège `None / NaN` aurait vidé la base en production.

— **Vérifier ses propres hypothèses :** les données par richesse n'étaient pas issues d'enquêtes ponctuelles, et cette vérification a ouvert la question de l'évolution des inégalités.

14 Feuille de route

Évolution	Apport
Historisation des révisions	Table d'historique (*slowly changing dimension* de type 2), au programme du projet 2
Chargement incrémental	Ne télécharger que les indicateurs mis à jour (champ `Date` de l'OMS)
Orchestration et CI/CD	Exécution planifiée après chaque publication, tests automatiques à chaque modification (projet 3)
Tableau de bord	Visualisation interactive pour l'équipe du client, connectée directement à PostgreSQL

A Requêtes SQL livrées

`00_schema.sql`, `01_views.sql`	Schéma (idempotent) et vues d'analyse
`q0_couverture_donnees.sql`	Couverture des données dans la région
`q1_cibles_2030.sql`	Situation par rapport aux cibles
`q2_rythme_2030.sql`, `q2b_…`	Trajectoire par pays et synthèse (méthode ARR)
`q3_inegalites_richesse.sql`, `q3b_…`	Écart pauvres / riches, et son évolution depuis 1990
`q4_moyens_resultats.sql`	Corrélations entre moyens et mortalité
`controle_recette.sql`	Contrôles de recette (C2, C3, C4)

B Décisions d'architecture (ADR)

ADR 001	PostgreSQL conteneurisé avec Docker Compose, secrets dans `.env`
ADR 002	Chargement par staging et UPSERT, clé métier complète `NULLS NOT DISTINCT`
ADR 003	Contrôles de qualité, quarantaine et calibrage des seuils

C Glossaire

ODD 3	Objectif de développement durable n°3 des Nations unies : « Bonne santé et bien-être »
Grain	Ce que représente une ligne d'une table
Staging	Zone de transit où le lot est chargé avant d'être fusionné dans les tables de référence
UPSERT	Insérer si la ligne est nouvelle, mettre à jour si elle existe (*update* + *insert*)
Quarantaine	Table qui conserve les lignes refusées par les contrôles, avec leur motif
ARR	*Annual Rate of Reduction* : taux annuel moyen de réduction d'un indicateur
Quintile de richesse	Cinquième de la population classée selon la richesse du ménage (Q1 : 20 % les plus pauvres)
Conteneur	Environnement isolé et reproductible qui exécute un logiciel, ici PostgreSQL